USQL Formal Grammar Specification

This document defines the complete formal grammar for USQL, UQA's SQL dialect. Only syntax that is actually implemented in the compiler (`uqa/sql/compiler.py`) is listed here. The notation follows ISO 14977 EBNF with the extensions noted below.

USQL Formal Grammar Specification
- Notation
- 1. Top-Level Statement
- 2. Data Definition Language (DDL)
  - 2.1 CREATE TABLE
  - 2.2 CREATE TABLE AS
  - 2.3 ALTER TABLE
  - 2.4 DROP TABLE
  - 2.5 TRUNCATE
  - 2.6 CREATE / DROP VIEW
  - 2.7 Indexes
  - 2.8 Sequences
  - 2.9 Foreign Data Wrappers
- 3. Data Manipulation Language (DML)
  - 3.1 INSERT
  - 3.2 UPDATE
  - 3.3 DELETE
- 4. Queries (SELECT)
  - 4.1 SELECT
  - 4.2 FROM Clause
  - 4.3 WHERE Clause
  - 4.4 GROUP BY
  - 4.5 ORDER BY
  - 4.6 LIMIT / OFFSET
  - 4.7 Set Operations
- 5. Common Table Expressions (WITH)
- 6. Window Functions
  - Ranking Functions
  - Value Functions
- 7. Aggregate Functions
  - Supported Aggregate Functions
- 8. Expressions
  - 8.1 Literals
  - 8.2 Column References
  - 8.3 Operators
  - 8.4 CASE Expression
  - 8.5 Type Cast
  - 8.6 Function Call
  - 8.7 Subquery Expression
  - 8.8 Parameter Reference
- 9. Data Types
- 10. Scalar Functions
  - 10.1 String Functions
  - 10.2 Regular Expression Functions
  - 10.3 Math Functions
  - 10.4 Trigonometric Functions
  - 10.5 Date/Time Functions

10.6 Conditional Functions
10.7 Type Inspection
10.8 Sequence Functions
10.9 Spatial Functions
10.10 UUID Functions
11. JSON/JSONB Functions
11.1 Construction
11.2 Access
11.3 Inspection
11.4 Containment Operators
11.5 Modification
11.6 Set-Returning JSON Functions
12. Array Functions
13. Cross-Paradigm Search Functions
13.1 Full-Text Search
13.2 Vector Similarity Search
13.3 Graph Traversal
13.4 Spatial Search
13.5 Hierarchical Filters
14. Probabilistic Fusion
15. Deep Learning Functions
15.1 Training
15.2 Prediction
15.3 Model Reference
15.4 Grid Graph Construction
16. Table-Returning Functions (FROM Clause)
17. Cypher / Graph Integration
18. Transactions
19. Prepared Statements
20. Utility Statements
20.1 EXPLAIN
20.2 ANALYZE
21. Information Schema
22. Hierarchical Functions (SELECT List)
Appendix A: Operator Precedence (highest to lowest)
Appendix B: Reserved Words

Notation

```
KEYWORD          Uppercase words are SQL keywords (case-insensitive)
name             Lowercase italic words are non-terminal symbols
'literal'        Single-quoted strings are literal tokens
|                Alternation
[ ... ]          Optional
{ ... }          Repetition (zero or more)
( ... )          Grouping
```

---

1. Top-Level Statement

```
statement
    = select_stmt
    | insert_stmt
    | update_stmt
    | delete_stmt
    | create_table_stmt
    | create_table_as_stmt
    | drop_table_stmt
    | alter_table_stmt
    | rename_stmt
    | truncate_stmt
    | create_view_stmt
    | drop_view_stmt
    | create_index_stmt
    | drop_index_stmt
    | create_sequence_stmt
    | alter_sequence_stmt
    | drop_sequence_stmt
    | create_foreign_server_stmt
    | create_foreign_table_stmt
    | drop_foreign_server_stmt
    | drop_foreign_table_stmt
    | explain_stmt
    | analyze_stmt
    | transaction_stmt
    | prepare_stmt
    | execute_stmt
    | deallocate_stmt
    ;
```

---

2. Data Definition Language (DDL)

2.1 CREATE TABLE

```
create_table_stmt
    = CREATE [ TEMPORARY | TEMP ] TABLE [ IF NOT EXISTS ] table_name
      '(' column_def { ',' column_def } { ',' table_constraint } ')'
    ;

column_def
    = column_name data_type { column_constraint }
    ;

column_constraint
    = PRIMARY KEY
    | NOT NULL
    | NULL
    | UNIQUE
    | DEFAULT expression
    | CHECK '(' expression ')'
    | REFERENCES table_name '(' column_name ')'
    ;

```

```
table_constraint
    = PRIMARY KEY '(' column_name { ',' column_name } ')'
    | UNIQUE '(' column_name { ',' column_name } ')'
    | CHECK '(' expression ')'
    | FOREIGN KEY '(' column_name ')' REFERENCES table_name '(' column_name ')'
    ;
```

2.2 CREATE TABLE AS

```
create_table_as_stmt
    = CREATE [ TEMPORARY | TEMP ] TABLE table_name AS select_stmt
    ;
```

2.3 ALTER TABLE

```
alter_table_stmt
    = ALTER TABLE table_name alter_action
    ;

alter_action
    = ADD [ COLUMN ] column_name data_type { column_constraint }
    | DROP [ COLUMN ] [ IF EXISTS ] column_name
    | RENAME [ COLUMN ] column_name TO column_name
    | RENAME TO table_name
    | ALTER [ COLUMN ] column_name SET DEFAULT expression
    | ALTER [ COLUMN ] column_name DROP DEFAULT
    | ALTER [ COLUMN ] column_name SET NOT NULL
    | ALTER [ COLUMN ] column_name DROP NOT NULL
    ;
```

2.4 DROP TABLE

```
drop_table_stmt
    = DROP TABLE [ IF EXISTS ] table_name { ',' table_name }
    ;
```

2.5 TRUNCATE

```
truncate_stmt
    = TRUNCATE [ TABLE ] table_name [ RESTART IDENTITY ]
    ;
```

2.6 CREATE / DROP VIEW

```
create_view_stmt
    = CREATE VIEW view_name AS select_stmt
    ;

drop_view_stmt
    = DROP VIEW [ IF EXISTS ] view_name
    ;
```

2.7 Indexes

```
create_index_stmt
    = CREATE [ UNIQUE ] INDEX [ index_name ] ON table_name
      [ USING index_method ]
      '(' index_column { ',' index_column } ')'
      [ WITH '(' index_parameter { ',' index_parameter } ')' ]
    ;

index_method
    = 'btree' | 'hnsw' | 'rtree'
    ;

index_column
    = column_name [ ASC | DESC ]
    ;

index_parameter
    = parameter_name '=' numeric_literal
    ;

drop_index_stmt
    = DROP INDEX [ IF EXISTS ] index_name
    ;
```

When `index_method` is `hnsw`, the column must be of type `VECTOR(n)`. Supported `index_parameter` names: `ef_construction` (default 200), `m` (default 16).

When `index_method` is `rtree`, the column must be of type `POINT`. Creates an SQLite R*Tree virtual table for O(log N) spatial range queries. No additional parameters are supported.

2.8 Sequences

```
create_sequence_stmt
    = CREATE SEQUENCE sequence_name
      [ START [ WITH ] integer ]
      [ INCREMENT [ BY ] integer ]
      [ MINVALUE integer ]
      [ MAXVALUE integer ]
      [ CYCLE | NO CYCLE ]
    ;

alter_sequence_stmt
    = ALTER SEQUENCE sequence_name
      [ RESTART [ WITH integer ] ]
```

```
      [ INCREMENT [ BY ] integer ]
      [ START [ WITH ] integer ]
    ;

drop_sequence_stmt
    = DROP SEQUENCE [ IF EXISTS ] sequence_name
    ;
```

2.9 Foreign Data Wrappers

```
create_foreign_server_stmt
    = CREATE FOREIGN SERVER server_name
      TYPE string_literal
      [ VERSION string_literal ]
      OPTIONS '(' option_pair { ',' option_pair } ')'
    ;

create_foreign_table_stmt
    = CREATE FOREIGN TABLE table_name
      '(' column_def { ',' column_def } ')'
      SERVER server_name
      [ OPTIONS '(' option_pair { ',' option_pair } ')' ]
    ;

option_pair
    = option_name string_literal
    ;

drop_foreign_server_stmt
    = DROP FOREIGN SERVER [ IF EXISTS ] server_name
    ;

drop_foreign_table_stmt
    = DROP FOREIGN TABLE [ IF EXISTS ] table_name
    ;
```

---

3. Data Manipulation Language (DML)

3.1 INSERT

```
insert_stmt
    = INSERT INTO table_name [ '(' column_name { ',' column_name } ')' ]
      insert_source
      [ on_conflict_clause ]
      [ returning_clause ]
    ;

insert_source
    = VALUES row_value { ',' row_value }
    | select_stmt
    ;

```

```
row_value
    = '(' expression { ',' expression } ')'
    ;

on_conflict_clause
    = ON CONFLICT '(' column_name { ',' column_name } ')'
      DO NOTHING
    | ON CONFLICT '(' column_name { ',' column_name } ')'
      DO UPDATE SET assignment { ',' assignment }
      [ WHERE expression ]
    ;

assignment
    = column_name '=' expression
    ;

returning_clause
    = RETURNING select_list
    ;
```

The `EXCLUDED` pseudo-table references the row proposed for insertion inside `ON CONFLICT DO UPDATE SET` expressions.

Expressions inside `row_value` may include `param_ref` (`$1`, `$2`, ...) for parameterized inserts. This is particularly efficient for vector embeddings, where parameter binding avoids the overhead of constructing `ARRAY[...]` string literals (approximately 65x faster).

3.2 UPDATE

```
update_stmt
    = UPDATE table_name [ [ AS ] alias ]
      SET assignment { ',' assignment }
      [ FROM from_item { ',' from_item } ]
      [ WHERE expression ]
      [ returning_clause ]
    ;
```

3.3 DELETE

```
delete_stmt
    = DELETE FROM table_name [ [ AS ] alias ]
      [ USING from_item { ',' from_item } ]
      [ WHERE expression ]
      [ returning_clause ]
    ;
```

---

4. Queries (SELECT)

4.1 SELECT

```
select_stmt
```

```
    = [ with_clause ] select_body
      { set_operation select_body }
      [ order_by_clause ]
      [ limit_clause ]
    ;

select_body
    = SELECT [ DISTINCT [ ON '(' expression { ',' expression } ')' ] ]
      select_list
      [ FROM from_item { ',' from_item } ]
      [ WHERE expression ]
      [ GROUP BY group_by_item { ',' group_by_item } ]
      [ HAVING expression ]
      [ WINDOW window_def { ',' window_def } ]
    ;

select_list
    = '*'
    | select_item { ',' select_item }
    ;

select_item
    = expression [ [ AS ] alias ]
    ;
```

4.2 FROM Clause

```
from_item
    = table_reference
    | joined_table
    | subquery_reference
    | lateral_reference
    | function_reference
    ;

table_reference
    = table_name [ [ AS ] alias ]
    ;

joined_table
    = from_item join_type JOIN from_item ON expression
    | from_item join_type JOIN from_item USING '(' column_name { ',' column_name } ')'
    | from_item CROSS JOIN from_item
    ;

join_type
    = [ INNER ]
    | LEFT [ OUTER ]
    | RIGHT [ OUTER ]
    | FULL [ OUTER ]
    ;

subquery_reference
```

```
    = '(' select_stmt ')' [ AS ] alias
    ;

lateral_reference
    = LATERAL '(' select_stmt ')' [ AS ] alias
    | from_item ',' LATERAL '(' select_stmt ')' [ AS ] alias
    ;

function_reference
    = function_call [ AS alias [ '(' column_def_list ')' ] ]
    ;
```

Multiple comma-separated items in `FROM` produce implicit cross joins.

4.3 WHERE Clause

```
where_expression
    = comparison_expr
    | boolean_expr
    | null_test
    | between_expr
    | in_expr
    | like_expr
    | exists_expr
    | sublink_expr
    | search_function
    | fusion_function
    ;

comparison_expr
    = expression comp_op expression
    ;

comp_op
    = '=' | '!=' | '<>' | '<' | '>' | '<=' | '>='
    ;

boolean_expr
    = expression AND expression
    | expression OR expression
    | NOT expression
    ;

null_test
    = expression IS NULL
    | expression IS NOT NULL
    ;

between_expr
    = expression [ NOT ] BETWEEN expression AND expression
    ;

in_expr
    = expression [ NOT ] IN '(' expression { ',' expression } ')'
```

```
    | expression [ NOT ] IN '(' select_stmt ')'
    ;

like_expr
    = expression [ NOT ] LIKE string_literal [ ESCAPE string_literal ]
    | expression [ NOT ] ILIKE string_literal [ ESCAPE string_literal ]
    ;

exists_expr
    = EXISTS '(' select_stmt ')'
    | NOT EXISTS '(' select_stmt ')'
    ;

sublink_expr
    = expression comp_op '(' select_stmt ')'
    ;
```

4.4 GROUP BY

```
group_by_item
    = column_name
    | column_alias
    | integer_literal                       (* ordinal reference *)
    | expression                            (* computed expression *)
    ;
```

4.5 ORDER BY

```
order_by_clause
    = ORDER BY sort_item { ',' sort_item }
    ;

sort_item
    = expression [ ASC | DESC ] [ NULLS FIRST | NULLS LAST ]
    ;
```

`expression` may be a column name, column alias, ordinal (integer), or arbitrary expression.

4.6 LIMIT / OFFSET

```
limit_clause
    = LIMIT expression [ OFFSET expression ]
    | OFFSET expression
    | FETCH FIRST expression ROWS ONLY
    | FETCH NEXT expression ROWS ONLY
    ;
```

4.7 Set Operations

```
set_operation
    = UNION [ ALL ]
    | INTERSECT [ ALL ]
    | EXCEPT [ ALL ]
    ;
```

---

5. Common Table Expressions (WITH)

```
with_clause
    = WITH [ RECURSIVE ] cte_def { ',' cte_def }
    ;

cte_def
    = cte_name [ '(' column_name { ',' column_name } ')' ] AS '(' select_stmt ')'
    ;
```

Recursive CTEs require `UNION ALL` between the base case and the recursive case. Termination occurs at fixpoint (when the recursive step produces no new rows). Maximum recursion depth defaults to 1000.

---

6. Window Functions

```
window_function
    = function_name '(' [ expression { ',' expression } ] ')' OVER window_spec
    | function_name '(' [ expression { ',' expression } ] ')' OVER window_name
    ;

window_spec
    = '(' [ PARTITION BY expression { ',' expression } ]
          [ ORDER BY sort_item { ',' sort_item } ]
          [ frame_clause ]
      ')'
    ;

frame_clause
    = ROWS BETWEEN frame_bound AND frame_bound
    | RANGE BETWEEN frame_bound AND frame_bound
    ;

frame_bound
    = UNBOUNDED PRECEDING
    | UNBOUNDED FOLLOWING
    | CURRENT ROW
    | integer_literal PRECEDING
    | integer_literal FOLLOWING
    ;

```

```
window_def
    = window_name AS window_spec
    ;
```

Ranking Functions

```
ROW_NUMBER() OVER (...)
RANK() OVER (...)
DENSE_RANK() OVER (...)
NTILE( integer ) OVER (...)
PERCENT_RANK() OVER (...)
CUME_DIST() OVER (...)
```

Value Functions

```
LAG( expression [, offset [, default ]] ) OVER (...)
LEAD( expression [, offset [, default ]] ) OVER (...)
FIRST_VALUE( expression ) OVER (...)
LAST_VALUE( expression ) OVER (...)
NTH_VALUE( expression , integer ) OVER (...)
```

Any aggregate function can also be used as a window function with an `OVER (...)` clause.

---

7. Aggregate Functions

```
aggregate_call
    = aggregate_name '(' [ DISTINCT ] expression { ',' expression } ')'
      [ agg_order_by ]
      [ filter_clause ]
    | aggregate_name '(' '*' ')' [ filter_clause ]
    ;

agg_order_by
    = ORDER BY sort_item { ',' sort_item }
    ;

filter_clause
    = FILTER '(' WHERE expression ')'
    ;
```

Supported Aggregate Functions

Category	Functions
Basic	`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
Collection	`STRING_AGG`, `ARRAY_AGG`, `BOOL_AND`, `BOOL_OR`
JSON	`JSON_OBJECT_AGG`, `JSONB_OBJECT_AGG`
Statistical	`STDDEV`, `STDDEV_POP`, `STDDEV_SAMP`, `VARIANCE`, `VAR_POP`, `VAR_SAMP`
Correlation	`CORR`, `COVAR_POP`, `COVAR_SAMP`
Regression	`REGR_SLOPE`, `REGR_INTERCEPT`, `REGR_R2`, `REGR_COUNT`, `REGR_AVGX`, `REGR_AVGY`, `REGR_SXX`, `REGR_SYY`, `REGR_SXY`
Deep Learning	`DEEP_LEARN`

`COUNT(*)` counts all rows. `COUNT(DISTINCT col)` counts distinct non-NULL values.

`STRING_AGG` and `ARRAY_AGG` accept an optional `ORDER BY` inside the aggregate call.

---

8. Expressions

8.1 Literals

```
literal
    = integer_literal
    | float_literal
    | string_literal
    | boolean_literal
    | null_literal
    | array_literal
    ;

integer_literal = digit { digit } ;
float_literal   = digit { digit } '.' digit { digit } ;
string_literal  = "'" { character } "'" ;
boolean_literal = TRUE | FALSE ;
null_literal    = NULL ;
array_literal   = ARRAY '[' expression { ',' expression } ']' ;
```

8.2 Column References

```
column_ref
    = column_name
    | table_name '.' column_name
    | '*'
    | table_name '.' '*'
    ;
```

8.3 Operators

```
expression
    = unary_expression
    | expression arithmetic_op expression
    | expression comp_op expression
    | expression string_op expression
    | expression boolean_op expression
    | expression json_op expression
    | expression IS [ NOT ] NULL
    | expression [ NOT ] BETWEEN expression AND expression
    | expression [ NOT ] IN '(' expression_list ')'
    | expression [ NOT ] IN '(' select_stmt ')'
    | expression [ NOT ] LIKE expression [ ESCAPE expression ]
    | expression [ NOT ] ILIKE expression [ ESCAPE expression ]
    | '(' expression ')'
    | '(' select_stmt ')'
    ;

unary_expression
    = [ '+' | '-' ] primary_expression
    | NOT expression
    ;

arithmetic_op = '+' | '-' | '*' | '/' | '%' ;
string_op     = '||' ;
boolean_op    = AND | OR ;

json_op
    = '->'                 (* field access, returns JSON *)
    | '->>'                (* field access, returns TEXT *)
    | '#>'                 (* path access, returns JSON *)
    | '#>>'                (* path access, returns TEXT *)
    | '@>'                 (* contains *)
    | '<@'                 (* contained by *)
    | '?'                  (* has key *)
    | '?|'                 (* has any key *)
    | '?&'                 (* has all keys *)
    ;
```

8.4 CASE Expression

```
case_expression
    = CASE { WHEN expression THEN expression } [ ELSE expression ] END
    | CASE expression { WHEN expression THEN expression } [ ELSE expression ] END
    ;
```

8.5 Type Cast

```
type_cast
    = CAST '(' expression AS data_type ')'
    | expression '::' data_type
    ;
```

8.6 Function Call

```
function_call
    = function_name '(' [ expression { ',' expression } ] ')'
    | function_name '(' '*' ')'
    ;
```

8.7 Subquery Expression

```
subquery_expression
    = '(' select_stmt ')'                          (* scalar subquery *)
    | expression [ NOT ] IN '(' select_stmt ')'
    | EXISTS '(' select_stmt ')'
    ;
```

8.8 Parameter Reference

```
param_ref = '$' integer_literal ;
```

Used in `PREPARE` statements and `engine.sql()` parameter binding.

---

9. Data Types

```
data_type
    = numeric_type
    | text_type
    | boolean_type
    | datetime_type
    | json_type
    | binary_type
    | special_type
    | array_type
    ;

numeric_type
    = INTEGER | INT | INT2 | INT4 | INT8
    | BIGINT | SMALLINT
    | SERIAL | BIGSERIAL
    | REAL | FLOAT | FLOAT4 | FLOAT8 | DOUBLE PRECISION
    | NUMERIC [ '(' precision ',' scale ')' ]
    | DECIMAL [ '(' precision ',' scale ')' ]
    ;

text_type
    = TEXT
    | VARCHAR [ '(' length ')' ]
    | CHARACTER VARYING [ '(' length ')' ]
    | CHAR [ '(' length ')' ]
    | CHARACTER [ '(' length ')' ]
    | NAME
```

```
    ;

boolean_type = BOOLEAN | BOOL ;

datetime_type
    = DATE
    | TIMESTAMP [ WITHOUT TIME ZONE ]
    | TIMESTAMPTZ
    | TIMESTAMP WITH TIME ZONE
    | INTERVAL
    ;

json_type = JSON | JSONB ;

binary_type = BYTEA ;

special_type
    = UUID
    | VECTOR '(' integer_literal ')'
    | POINT
    ;

array_type = data_type '[]' ;
```

`VECTOR(n)` defines an n-dimensional vector column. Vectors are stored in the document store as JSON arrays. For approximate nearest neighbor search, create an HNSW index with `CREATE INDEX ... USING hnsw (column)`. Without an index, `knn_match()` uses brute-force exact cosine similarity.

`POINT` defines a 2-D coordinate column storing `[longitude, latitude]`. For O(log N) spatial range queries, create an R*Tree index with `CREATE INDEX ... USING rtree (column)`. Without an index, `spatial_within()` falls back to brute-force Haversine scan.

---

10. Scalar Functions

10.1 String Functions

```
UPPER( text ) -> text
LOWER( text ) -> text
INITCAP( text ) -> text
LENGTH( text ) -> integer
CHAR_LENGTH( text ) -> integer
CHARACTER_LENGTH( text ) -> integer
OCTET_LENGTH( text ) -> integer
SUBSTRING( text, start [, length] ) -> text
SUBSTR( text, start [, length] ) -> text
POSITION( substring IN text ) -> integer
STRPOS( text, substring ) -> integer
TRIM( text ) -> text
BTRIM( text [, characters] ) -> text
LTRIM( text [, characters] ) -> text
RTRIM( text [, characters] ) -> text
```

```
LPAD( text, length [, fill] ) -> text
RPAD( text, length [, fill] ) -> text
REPEAT( text, count ) -> text
REVERSE( text ) -> text
REPLACE( text, from, to ) -> text
CONCAT( value, ... ) -> text
CONCAT_WS( separator, value, ... ) -> text
SPLIT_PART( text, delimiter, index ) -> text
LEFT( text, count ) -> text
RIGHT( text, count ) -> text
TRANSLATE( text, from_chars, to_chars ) -> text
ASCII( text ) -> integer
CHR( code ) -> text
STARTS_WITH( text, prefix ) -> boolean
OVERLAY( text, replacement, start [, length] ) -> text
FORMAT( format_string, ... ) -> text
ENCODE( data, format ) -> text
DECODE( text, format ) -> bytea
MD5( text ) -> text
text || text -> text
```

10.2 Regular Expression Functions

```
REGEXP_MATCH( text, pattern [, flags] ) -> text[]
REGEXP_MATCHES( text, pattern [, flags] ) -> SETOF text[]
REGEXP_REPLACE( text, pattern, replacement [, flags] ) -> text
REGEXP_SPLIT_TO_ARRAY( text, pattern [, flags] ) -> text[]
```

Flags: `g` (global), `i` (case-insensitive), `s` (dotall).

10.3 Math Functions

```
ABS( numeric ) -> numeric
SIGN( numeric ) -> integer
ROUND( numeric [, digits] ) -> numeric
TRUNC( numeric [, digits] ) -> numeric
CEIL( numeric ) -> integer
CEILING( numeric ) -> integer
FLOOR( numeric ) -> integer
POWER( base, exponent ) -> numeric
POW( base, exponent ) -> numeric
SQRT( numeric ) -> numeric
CBRT( numeric ) -> numeric
EXP( numeric ) -> numeric
LN( numeric ) -> numeric
LOG( numeric ) -> numeric
LOG10( numeric ) -> numeric
LOG( base, value ) -> numeric
DIV( dividend, divisor ) -> integer
MOD( dividend, divisor ) -> numeric
GCD( a, b ) -> integer
LCM( a, b ) -> integer
PI() -> numeric
```

```
RANDOM() -> numeric
WIDTH_BUCKET( value, low, high, buckets ) -> integer
MIN_SCALE( numeric ) -> integer
TRIM_SCALE( numeric ) -> numeric
```

10.4 Trigonometric Functions

```
SIN( radians ) -> numeric
COS( radians ) -> numeric
TAN( radians ) -> numeric
ASIN( value ) -> numeric
ACOS( value ) -> numeric
ATAN( value ) -> numeric
ATAN2( y, x ) -> numeric
DEGREES( radians ) -> numeric
RADIANS( degrees ) -> numeric
```

10.5 Date/Time Functions

```
NOW() -> timestamp
CURRENT_TIMESTAMP -> timestamp
CURRENT_DATE -> date
CURRENT_TIME -> time
CLOCK_TIMESTAMP() -> timestamp
TIMEOFDAY() -> text
EXTRACT( field FROM timestamp ) -> numeric
DATE_PART( field, timestamp ) -> numeric
DATE_TRUNC( precision, timestamp ) -> timestamp
TO_CHAR( timestamp, format ) -> text
TO_DATE( text, format ) -> date
TO_TIMESTAMP( text, format ) -> timestamp
TO_NUMBER( text ) -> numeric
MAKE_DATE( year, month, day ) -> date
MAKE_TIMESTAMP( year, month, day, hour, minute, second ) -> timestamp
MAKE_INTERVAL( [years, months, weeks, days, hours, minutes, seconds] ) -> interval
AGE( timestamp [, timestamp] ) -> interval
OVERLAPS( start1, end1, start2, end2 ) -> boolean
ISFINITE( value ) -> boolean
```

`EXTRACT` / `DATE_PART` fields: `year`, `month`, `day`, `hour`, `minute`, `second`, `epoch`, `dow`, `doy`, `quarter`, `week`.

`DATE_TRUNC` precision values: `year`, `month`, `day`, `hour`, `minute`, `second`.

10.6 Conditional Functions

```
COALESCE( value, value, ... ) -> value
NULLIF( value1, value2 ) -> value
GREATEST( value, value, ... ) -> value
LEAST( value, value, ... ) -> value
```

10.7 Type Inspection

```
TYPEOF( value ) -> text
CAST( value AS type ) -> value
value :: type -> value
```

10.8 Sequence Functions

```
NEXTVAL( sequence_name ) -> integer
CURRVAL( sequence_name ) -> integer
SETVAL( sequence_name, value ) -> integer
```

10.9 Spatial Functions

```
POINT( x, y ) -> point
ST_DISTANCE( point, point ) -> numeric
ST_WITHIN( point, point, distance ) -> boolean
ST_DWITHIN( point, point, distance ) -> boolean
```

`POINT(x, y)` constructs a point value from longitude and latitude. `ST_DISTANCE` computes the Haversine great-circle distance in meters. `ST_WITHIN` and `ST_DWITHIN` are boolean distance predicates (aliases for each other).

10.10 UUID Functions

```
GEN_RANDOM_UUID() -> uuid
```

---

11. JSON/JSONB Functions

11.1 Construction

```
JSON_BUILD_OBJECT( key, value, ... ) -> json
JSONB_BUILD_OBJECT( key, value, ... ) -> jsonb
JSON_BUILD_ARRAY( value, ... ) -> json
JSONB_BUILD_ARRAY( value, ... ) -> jsonb
TO_JSON( value ) -> json
TO_JSONB( value ) -> jsonb
ROW_TO_JSON( record ) -> json
```

11.2 Access

```
json -> key -> json
json ->> key -> text
json -> index -> json
json ->> index -> text
json #> '{key1,key2}' -> json
json #>> '{key1,key2}' -> text
JSON_EXTRACT_PATH( json, key1, key2, ... ) -> json
JSON_EXTRACT_PATH_TEXT( json, key1, key2, ... ) -> text
JSONB_EXTRACT_PATH( jsonb, key1, key2, ... ) -> jsonb
JSONB_EXTRACT_PATH_TEXT( jsonb, key1, key2, ... ) -> text
```

11.3 Inspection

```
JSON_TYPEOF( json ) -> text
JSONB_TYPEOF( jsonb ) -> text
JSON_ARRAY_LENGTH( json ) -> integer
JSONB_ARRAY_LENGTH( jsonb ) -> integer
```

`JSON_TYPEOF` returns: `object`, `array`, `string`, `number`, `boolean`, `null`.

11.4 Containment Operators

```
jsonb @> jsonb -> boolean
jsonb <@ jsonb -> boolean
jsonb ? text -> boolean
jsonb ?| text[] -> boolean
jsonb ?& text[] -> boolean
```

11.5 Modification

```
JSONB_SET( target, path, new_value [, create_if_missing] ) -> jsonb
JSONB_INSERT( target, path, new_value ) -> jsonb
JSONB_STRIP_NULLS( jsonb ) -> jsonb
JSON_STRIP_NULLS( json ) -> json
```

11.6 Set-Returning JSON Functions

```
JSON_EACH( json ) -> SETOF (key text, value json)
JSONB_EACH( jsonb ) -> SETOF (key text, value jsonb)
JSON_EACH_TEXT( json ) -> SETOF (key text, value text)
JSONB_EACH_TEXT( jsonb ) -> SETOF (key text, value text)
JSON_ARRAY_ELEMENTS( json ) -> SETOF json
JSONB_ARRAY_ELEMENTS( jsonb ) -> SETOF jsonb
JSON_ARRAY_ELEMENTS_TEXT( json ) -> SETOF text
JSONB_ARRAY_ELEMENTS_TEXT( jsonb ) -> SETOF text
JSON_OBJECT_KEYS( json ) -> SETOF text
JSONB_OBJECT_KEYS( jsonb ) -> SETOF text
```

---

12. Array Functions

```
ARRAY_LENGTH( array, dimension ) -> integer
ARRAY_UPPER( array, dimension ) -> integer
ARRAY_LOWER( array, dimension ) -> integer
ARRAY_CAT( array, array ) -> array
ARRAY_APPEND( array, element ) -> array
ARRAY_REMOVE( array, element ) -> array
CARDINALITY( array ) -> integer
```

---

13. Cross-Paradigm Search Functions

These functions appear in WHERE clauses and produce relevance scores accessible via the `_score` pseudo-column.

13.1 Full-Text Search

```
text_match_call     = TEXT_MATCH '(' column_name ',' string_literal ')' ;
bayesian_match_call = BAYESIAN_MATCH '(' column_name ',' string_literal ')' ;
```

`TEXT_MATCH` scores with BM25. `BAYESIAN_MATCH` applies Bayesian calibration to produce $P(\text{relevant}) \in [0, 1]$.

13.2 Vector Similarity Search

```
knn_match_call
    = KNN_MATCH '(' column_name ',' vector_arg ',' integer_literal ')'
    ;

vector_exclude_call
    = VECTOR_EXCLUDE '(' column_name ',' vector_arg ',' vector_arg ','
                         integer_literal ',' numeric_literal ')'
    ;

vector_arg
    = array_literal
    | param_ref
    ;
```

`KNN_MATCH` returns the top-k nearest neighbors by cosine similarity. Uses the HNSW index if one exists on the column, otherwise falls back to brute-force exact scan.

13.3 Graph Traversal

```
traverse_match_call
    = TRAVERSE_MATCH '(' integer_literal ',' string_literal ',' integer_literal ')'
    ;
```

13.4 Spatial Search

```
spatial_within_call
    = SPATIAL_WITHIN '(' column_name ',' point_arg ',' numeric_literal ')'
    ;

point_arg
    = POINT '(' numeric_literal ',' numeric_literal ')'
    | param_ref
    ;
```

`SPATIAL_WITHIN` returns all points within a given distance (meters) of a center point, scored by proximity. Uses the R*Tree index if available, otherwise falls back to brute-force Haversine scan.

13.5 Hierarchical Filters

```
path_filter_call
    = PATH_FILTER '(' string_literal ',' expression ')'
    | PATH_FILTER '(' string_literal ',' string_literal ',' expression ')'
    ;
```

---

14. Probabilistic Fusion

Fusion meta-functions combine multiple search signals in the WHERE clause. Each signal argument must be a search function call.

```
fusion_call
    = FUSE_LOG_ODDS '(' signal { ',' signal } [ ',' numeric_literal ] [ ',' gating_literal ] ')'
    | FUSE_PROB_AND '(' signal { ',' signal } ')'
    | FUSE_PROB_OR '(' signal { ',' signal } ')'
    | FUSE_PROB_NOT '(' signal ')'
    ;

signal
    = text_match_call
    | bayesian_match_call
    | knn_match_call
    | traverse_match_call
    | spatial_within_call
    ;

gating_literal
    = '''relu'''
    | '''swish'''
    ;
```

For `FUSE_LOG_ODDS`, the optional trailing arguments are:

- `numeric_literal`: confidence alpha (default 0.5)
- `gating_literal`: activation gating (`'relu'` or `'swish'`, default none)

Inside a fusion context, `TEXT_MATCH` is automatically promoted to Bayesian BM25 calibration so all signals produce probabilities in $(0, 1)$.

---

15. Deep Learning Functions

These functions provide neural network training, prediction, and graph construction for deep learning over posting-list data.

15.1 Training

```
deep_learn_call
    = DEEP_LEARN '(' model_name ',' label_col ',' embedding_col ',' edge_label ','
                     layer_spec { ',' layer_spec }
                     { ',' named_option } ')'
    ;

layer_spec
    = convolve_spec
    | pool_spec
    | flatten_spec
    | global_pool_spec
    | dense_spec
    | softmax_spec
    | attention_spec
    ;

convolve_spec
    = CONVOLVE '(' [ 'n_channels' '=>' integer ] [ ',' 'seed' '=>' integer ]
                   [ ',' 'init' '=>' string_literal ] ')'
    ;

pool_spec
    = POOL '(' string_literal ',' integer ')'
    ;

flatten_spec
    = FLATTEN '(' ')'
    ;

global_pool_spec
    = GLOBAL_POOL '(' [ string_literal ] ')'
    ;

dense_spec
    = DENSE '(' 'output_channels' '=>' integer ')'
    ;

softmax_spec
    = SOFTMAX '(' ')'
    ;

attention_spec
```

```
    = ATTENTION '(' [ 'n_heads' '=>' integer ] [ ',' 'mode' '=>' string_literal ] ')'
    ;

named_option
    = 'gating' '=>' string_literal
    | 'lambda' '=>' numeric_literal
    | 'l1_ratio' '=>' numeric_literal
    | 'prune_ratio' '=>' numeric_literal
    ;
```

`DEEP_LEARN` trains a multi-layer neural network model. The `model_name` identifies the trained model for subsequent prediction. `label_col` and `embedding_col` reference the target and feature columns. `edge_label` specifies the graph edge label used for message passing in convolutional layers. Layer specs are composed in order — the first layer receives the input features and the last layer produces the output.

`CONVOLVE` applies a graph convolution with optional channel count and random seed. `POOL` applies a pooling operation (`'mean'` or `'max'`) with a given kernel size. `FLATTEN` reshapes multi-dimensional features into a single vector. `DENSE` applies a fully connected layer with the specified output dimension. `SOFTMAX` normalizes output scores into a probability distribution. `ATTENTION` applies self-attention (Theorem 8.3, Paper 4) with optional multi-head parallelism and mode selection: `'content'` (Q=K=V=X), `'random_qk'` (random Q,K projections, V=X), or `'learned_v'` (random Q,K, supervised V projection via ridge regression).

The optional `named_option` arguments control training behavior: `gating` selects an activation function (`'relu'` or `'swish'`), `lambda` sets the ridge regression regularization coefficient, `l1_ratio` enables elastic net regularization via proximal gradient descent (ISTA), and `prune_ratio` enables post-training magnitude pruning of the smallest weights by percentile.

15.2 Prediction

```
deep_predict_call
    = DEEP_PREDICT '(' model_name ',' expression ')'
    ;
```

`DEEP_PREDICT` applies a previously trained model to the given input expression and returns the predicted label or score.

15.3 Model Reference

```
model_call
    = MODEL '(' model_name ',' expression ')'
    ;
```

`MODEL` retrieves a named model and applies it to an expression. This is the general-purpose inference entry point.

15.4 Grid Graph Construction

```
build_grid_graph_call
    = BUILD_GRID_GRAPH '(' table_name ',' integer ',' integer ',' edge_label ')'
    ;
```

`BUILD_GRID_GRAPH` constructs a regular grid graph over the specified table with the given number of rows and columns. Each cell becomes a vertex and edges are created between adjacent cells using the specified `edge_label`.

---

16. Table-Returning Functions (FROM Clause)

```
generate_series_call
    = GENERATE_SERIES '(' start ',' stop [ ',' step ] ')'
    ;

regexp_split_to_table_call
    = REGEXP_SPLIT_TO_TABLE '(' text ',' pattern [ ',' flags ] ')'
    ;

traverse_call
    = TRAVERSE '(' start_id ',' label ',' max_hops [ ',' table_name ] ')'
    ;

rpq_call
    = RPQ '(' path_expression ',' start_id [ ',' graph_source ] ')'
    ;

pagerank_call
    = PAGERANK '(' [ damping [ ',' max_iterations [ ',' tolerance ] ] ] [ ',' graph_source ] ')'
    ;

hits_call
    = HITS '(' [ max_iterations [ ',' tolerance ] ] [ ',' graph_source ] ')'
    ;

betweenness_call
    = BETWEENNESS '(' [ graph_source ] ')'
    ;

weighted_rpq_call
    = WEIGHTED_RPQ '(' path_expression ',' start_id ',' weight_property
                       [ ',' aggregate_fn [ ',' threshold ] ] ')'
    ;

progressive_fusion_call
    = PROGRESSIVE_FUSION '(' signal { ',' signal } ',' integer_literal
                             { ',' signal { ',' signal } ',' integer_literal }
                             [ ',' numeric_literal ] [ ',' gating_literal ] ')'
    ;
```

```

graph_add_vertex_call
    = GRAPH_ADD_VERTEX '(' vertex_id ',' label ',' table_name [ ',' properties ] ')'
    ;

graph_add_edge_call
    = GRAPH_ADD_EDGE '(' edge_id ',' source_id ',' target_id ','
                         label ',' table_name [ ',' properties ] ')'
    ;

graph_source
    = string_literal        (* table name or graph name (direct, no prefix required) *)
    ;
```

Path expressions support bounded repetition: `'knows{2,4}'` matches paths of 2 to 4 hops.

`PAGERANK`, `HITS`, `BETWEENNESS` work as both FROM-clause table sources and WHERE-clause scored signals. The optional `graph_source` argument accepts a graph name directly (e.g., `'social'`) or a table name. The `graph:` prefix is accepted for backward compatibility but not required.

`WEIGHTED_RPQ` evaluates a regular path query tracking cumulative edge weight. `aggregate_fn` is `'sum'` (default), `'max'`, or `'min'`. When `threshold` is provided, only paths with cumulative weight exceeding the threshold are returned.

`PROGRESSIVE_FUSION` implements cascading multi-stage WAND fusion. Signals are grouped into stages separated by integer cutoffs. Each stage narrows the candidate set via top-k pruning.

`GRAPH_ADD_VERTEX` and `GRAPH_ADD_EDGE` add vertices and edges to a table's per-table graph store. Properties are specified as comma-separated `key=value` pairs in a single string.

`GENERATE_SERIES` produces integer or timestamp series.

---

17. Cypher / Graph Integration

```
cypher_call
    = CYPHER '(' cypher_query_string [ ',' graph_name ] ')'
    ;

create_graph_call
    = CREATE_GRAPH '(' graph_name ')'
    ;

drop_graph_call
    = DROP_GRAPH '(' graph_name ')'
    ;
```

These are used in the FROM clause:

```
SELECT * FROM cypher('MATCH (n) RETURN n.name') AS (name agtype);
SELECT * FROM cypher('MATCH (n) RETURN n', 'my_graph') AS (n agtype);
SELECT * FROM create_graph('social');
SELECT * FROM drop_graph('social');
```

See the USQL Language Reference for the full openCypher clause and pattern syntax.

---

18. Transactions

```
transaction_stmt
    = BEGIN [ TRANSACTION ]
    | COMMIT [ TRANSACTION ]
    | ROLLBACK [ TRANSACTION ]
    | SAVEPOINT savepoint_name
    | RELEASE SAVEPOINT savepoint_name
    | ROLLBACK TO SAVEPOINT savepoint_name
    ;
```

---

19. Prepared Statements

```
prepare_stmt
    = PREPARE statement_name [ '(' data_type { ',' data_type } ')' ] AS statement
    ;

execute_stmt
    = EXECUTE statement_name [ '(' expression { ',' expression } ')' ]
    ;

deallocate_stmt
    = DEALLOCATE statement_name
    | DEALLOCATE ALL
    ;
```

---

20. Utility Statements

20.1 EXPLAIN

```
explain_stmt
    = EXPLAIN select_stmt
    | EXPLAIN ANALYZE select_stmt
    | EXPLAIN '(' option { ',' option } ')' select_stmt
    ;

option
    = ANALYZE boolean_literal
    | VERBOSE boolean_literal
    ;
```

20.2 ANALYZE

```
analyze_stmt
    = ANALYZE [ table_name ]
    ;
```

Collects per-column statistics: distinct count, NULL count, min/max values, equi-depth histograms, most common values and frequencies. Used by the query optimizer for cardinality estimation.

---

21. Information Schema

```
information_schema_query
    = SELECT ... FROM information_schema '.' view_name [ WHERE ... ]
    ;
```

Supported views:

View	Columns
`information_schema.tables`	`table_catalog`, `table_schema`, `table_name`, `table_type`, `is_insertable_into`, `is_typed`, `commit_action`
`information_schema.columns`	`table_catalog`, `table_schema`, `table_name`, `column_name`, `ordinal_position`, `column_default`, `is_nullable`, `data_type`, `character_maximum_length`, `numeric_precision`, `numeric_scale`

---

22. Hierarchical Functions (SELECT List)

```
path_value_call = PATH_VALUE '(' path_string ')' ;
path_agg_call   = PATH_AGG '(' path_string ',' agg_name ')' ;
```

`PATH_VALUE` navigates a dot-delimited path in a nested document and returns the value found. `PATH_AGG` collects array element values at the path and applies an aggregation (`sum`, `count`, `avg`, `min`, `max`).

Appendix A: Operator Precedence (highest to lowest)

Precedence	Operator	Associativity
1	`::`	left
2	`->`, `->>`, `#>`, `#>>`	left
3	unary `-`, unary `+`	right
4	`*`, `/`, `%`	left
5	`+`, `-`	left
6	`||`	left
7	`@>`, `<@`, `?`, `?|`, `?&`	left
8	`=`, `!=`, `<>`, `<`, `>`, `<=`, `>=`	left
9	`IS`, `IS NOT`, `IN`, `LIKE`, `ILIKE`, `BETWEEN`	left
10	`NOT`	right
11	`AND`	left
12	`OR`	left

Appendix B: Reserved Words

The following words are treated as reserved keywords and cannot be used as unquoted identifiers:

```
ALL, ALTER, ANALYZE, AND, ANY, ARRAY, AS, ASC, BEGIN, BETWEEN, BIGINT,
BIGSERIAL, BOOLEAN, BY, CASCADE, CASE, CAST, CHECK, COLUMN, COMMIT,
CONFLICT, CREATE, CROSS, CURRENT, DATE, DEALLOCATE, DECIMAL, DEFAULT,
DELETE, DESC, DISTINCT, DO, DROP, ELSE, END, EXCEPT, EXCLUDED, EXECUTE,
EXISTS, EXPLAIN, FALSE, FETCH, FILTER, FIRST, FLOAT, FOLLOWING, FOR,
FOREIGN, FROM, FULL, GROUP, HAVING, IF, ILIKE, IN, INDEX, INNER,
INSERT, INTEGER, INTERSECT, INTERVAL, INTO, IS, JOIN, JSON, JSONB, KEY,
LAST, LATERAL, LEFT, LIKE, LIMIT, NOT, NULL, NULLS, NUMERIC, OFFSET,
ON, ONLY, OR, ORDER, OUTER, OVER, PARTITION, POINT, PRECEDING, PREPARE,
PRIMARY, RANGE, REAL, RECURSIVE, REFERENCES, RELEASE, RENAME, RESTART,
RETURNING, RIGHT, ROLLBACK, ROWS, SAVEPOINT, SELECT, SEQUENCE, SERIAL,
SERVER, SET, SMALLINT, TABLE, TEMP, TEMPORARY, TEXT, THEN, TIMESTAMP,
TIMESTAMPTZ, TO, TRANSACTION, TRUE, TRUNCATE, UNBOUNDED, UNION, UNIQUE,
UPDATE, USING, UUID, VACUUM, VALUES, VARCHAR, VECTOR, VIEW, WHEN,
WHERE, WINDOW, WITH
```

Identifiers that collide with reserved words must be double-quoted: `"select"`, `"table"`.